

Один считыватель с несколькими процессорами и записывающими устройствами в Spring Batch

Последнее обновление: 24 июня 2025 г.



Автор: Винн Тео (<https://www.baeldung.com/author/wynnteo>)



Проверено: Эриком Мартином (<https://www.baeldung.com/editor/eric-editor>)
Весна (<https://www.baeldung.com/category/spring>) +
Весенняя партия (<https://www.baeldung.com/tag/spring-batch>)



Традиционные методы поиска по ключевым словам основаны на точном совпадении слов, **что часто приводит к нерелевантным результатам** в зависимости от формулировки запроса пользователя.

Для сравнения: использование векторного хранилища позволяет нам представлять данные в виде векторных вложений, основанных на значимых взаимосвязях. Затем мы можем сравнить смысл запроса пользователя с сохранённым контентом и **получить более релевантные результаты с учётом контекста**.

Узнайте, как **создать интеллектуального чат-бота с помощью MongoDB Atlas, Langchain4j и Spring Boot**:

1. Введение

В этом руководстве мы рассмотрим, как реализовать задание Spring Batch (/introduction-to-spring-batch) с одним считывателем, несколькими процессорами и несколькими записывающими устройствами. Такой подход полезен, когда нам нужно один раз прочитать данные, обработать их разными способами, а затем записать результаты в несколько мест назначения.

2. Настройка проекта Spring Batch

Прежде чем мы начнём, нам нужно включить зависимости Spring Boot Starter Batch (<https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-batch>) и Spring Boot Starter Data JPA (<https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-data-jpa>) в наш файл *pom.xml*:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-batch</artifactId>
  <version>3.5.0</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
  <version>3.4.2</version>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

Эти зависимости включают Spring Batch для обработки заданий, Spring Data JPA (/the-persistence-layer-with-spring-data-jpa) для работы с базами данных и H2 в качестве базы данных в оперативной памяти для целей разработки.

2.1. Подготовка входного CSV-файла

Прежде чем внедрять пакетные компоненты, нам нужно обработать примерные данные. Давайте создадим простой CSV-файл (/java-csv) с именем *customers.csv* со следующим содержимым:

```
id,name,email,type
1,John,john@example.com,A
2,Alice,alice@example.com,B
3,Bob,bob@example.com,A
4,Eve,eve@example.com,B
```



Этот файл содержит записи о клиентах с четырьмя полями: уникальный идентификатор, имя, адрес электронной почты и обозначение типа, которое определит наш путь обработки данных. Мы сохраним этот файл в каталоге *src/main/resources* нашего проекта.

2.2. Создание модели данных

Для нашего пакетного задания требуется класс Java, представляющий данные о клиентах из нашего CSV-файла. Давайте создадим класс *Customer*, который будет соответствовать нашей таблице в базе данных:

```
@Entity
public class Customer {
    @Id
    private Long id;
    private String name;
    private String email;
    private String type;

    // Constructors, getters, and setters
}
```

3. Реализация средства чтения CSV-файлов

Теперь мы можем создать компонент, который считывает записи из нашего CSV-файла. Spring Batch обеспечивает отличную поддержку чтения обычных файлов с помощью класса *FlatFileItemReader*.

```
@Bean
public FlatFileItemReader<Customer> customerReader() {
    return new FlatFileItemReaderBuilder<Customer>()
        .name("customerItemReader")
        .resource(new ClassPathResource("customers.csv"))
        .delimited()
        .names("id", "name", "email", "type")
        .fieldSetMapper(new BeanWrapperFieldSetMapper<Customer>() {{
            setTargetType(Customer.class);
        }})
        .build();
}
```

Эта конфигурация создаёт средство чтения, которое построчно анализирует наш CSV-файл, сопоставляя каждую запись с объектом *Customer*. **Средство чтения автоматически открывает и закрывает файл и обрабатывает данные порциями для экономии памяти.**

Имена полей, указанные в методе *names()*, должны совпадать как с заголовком CSV, так и со свойствами класса *Customer*.

4. Создание условных обработчиков

Мы создадим два отдельных процессора и механизм маршрутизации для выбора между ними. **Каждый из этих процессоров реализует интерфейс Spring Batch *ItemProcessor*, который определяет один метод *process()*, используемый для преобразования входных данных перед их записью:**

(/)

```
public class TypeAProcessor implements ItemProcessor<Customer, Customer> {  
    @Override  
    public Customer process(Customer customer) {  
        customer.setName(customer.getName().toUpperCase());  
        customer.setEmail("A_" + customer.getEmail());  
        return customer;  
    }  
}  
  
public class TypeBProcessor implements ItemProcessor<Customer, Customer> {  
    @Override  
    public Customer process(Customer customer) {  
        customer.setName(customer.getName().toLowerCase());  
        customer.setEmail("B_" + customer.getEmail());  
        return customer;  
    }  
}
```

TypeAProcessor обрабатывает клиентов типа *A*, преобразуя их имена в верхний регистр и добавляя префикс к адресам электронной почты. **Метод *process()* принимает *Customer* объект, преобразует его и возвращает изменённую версию.**

Для клиентов типа *B* *TypeBProcessor* применяет другие преобразования, переводя имена в нижний регистр и используя другой префикс *email*. **Оба процессора реализуют один и тот же *ItemProcessor* интерфейс, что делает их взаимозаменяемыми в нашем конвейере обработки.**

5. Реализация маршрутизатора процессора

Чтобы связать наши процессоры с соответствующими записями, нам нужен механизм маршрутизации, который анализирует поле типа каждого клиента:

```
public class CustomerProcessorRouter implements ItemProcessor<Customer,
Customer> {
    private final TypeAProcessor typeAProcessor;
    private final TypeBProcessor typeBProcessor;

    public CustomerProcessorRouter(TypeAProcessor typeAProcessor,
        TypeBProcessor typeBProcessor) {
        this.typeAProcessor = typeAProcessor;
        this.typeBProcessor = typeBProcessor;
    }

    @Override
    public Customer process(Customer customer) throws Exception {
        if ("A".equals(customer.getType())) {
            return typeAProcessor.process(customer);
        } else if ("B".equals(customer.getType())) {
            return typeBProcessor.process(customer);
        }
        return customer;
    }
}
```

Наш класс-маршрутизатор анализирует каждый входящий объект *Customer* и перенаправляет его соответствующему обработчику на основе поля *type*. Благодаря такой структуре наша логика обработки чётко разделена, но при этом в определении задачи сохраняется единый этап обработки.

6. Настройка нескольких устройств записи

После обработки данных в зависимости от их типа мы хотим записать результаты в несколько мест назначения. Мы реализуем запись в базу данных и в обычный файл.

6.1. Настройка записи в базу данных

Начнём с создания компонента записи в базу данных, который будет обрабатывать все операции JPA:

```
@Bean
public JpaItemWriter<Customer> dbWriter(EntityManagerFactory
entityManagerFactory) {
    JpaItemWriter<Customer> writer = new JpaItemWriter<>();
    writer.setEntityManagerFactory(entityManagerFactory);
    return writer;
}
```

Этот модуль использует JPA для сохранения обработанных объектов *Customer* в базе данных. **Во время выполнения задания этот *JpaItemWriter* будет сохранять обработанные *Customer* объекты в настроенной базе данных, выполняя все необходимые операции JPA, включая вставку и обновление.**

6.2. Настройка записи в плоский файл

Для дополнительного места назначения вывода мы реализуем запись в обычный файл, который генерирует файл CSV:

```
@Bean
public FlatFileItemWriter<Customer> fileWriter() {
    return new FlatFileItemWriterBuilder<Customer>()
        .name("customerItemWriter")
        .resource(new FileSystemResource("output/processed_customers.txt"))
        .delimited()
        .delimiter(",")
        .names("id", "name", "email", "type")
        .build();
}
```

FlatFileItemWriter настроен на использование разделителей в виде запятых и содержит явные имена полей, соответствующие свойствам объекта *Customer*. **Во время выполнения задания этот модуль создаст структурированный CSV-файл, содержащий все обработанные записи о клиентах в указанном формате.**

6.3. Объединение компонентов в составном средстве записи

Чтобы одновременно записывать данные в оба места назначения, мы будем использовать *CompositeItemWriter* из Spring Batch:

```
@Bean
public CompositeItemWriter<Customer> compositeWriter(
    JpaItemWriter<Customer> dbWriter,
    FlatFileItemWriter<Customer> fileWriter) {
    CompositeItemWriter<Customer> writer = new CompositeItemWriter<>();
    writer.setDelegates(List.of(dbWriter, fileWriter));
    return writer;
}
```

Этот составной модуль записи выступает в роли делегата для модулей записи, гарантируя, что каждый обработанный элемент будет записан во все места назначения. Порядок делегатов определяет последовательность записи.

7. Настройка этапа и задания

Теперь давайте соберём всё воедино, создав шаг и конфигурацию задания:

```
@Bean
public Job processCustomersJob(JobBuilderFactory jobs,
    StepBuilderFactory steps,
    FlatFileItemReader<Customer> reader,
    CustomerProcessorRouter processor,
    CompositeItemWriter<Customer> writer) {

    Step step = steps.get("processCustomersStep")
        .<Customer, Customer>chunk(10)
        .reader(reader)
        .processor(processor)
        .writer(writer)
        .build();

    return jobs.get("customerProcessingJob")
        .start(step)
        .build();
}
```

Эта конфигурация задания определяет один шаг, который считывает данные о клиентах блоками по 10, обрабатывает каждый блок с помощью нашего маршрутизатора и записывает результаты как в базу данных, так и в обычный файл.

8. Запуск и тестирование задания

Чтобы убедиться, что наше пакетное задание выполняется должным образом, давайте напишем интеграционный тест, который запустит задание и проверит результаты в базе данных и в выходном файле для разных типов клиентов:

```
List<Customer> dbCustomers = jdbcTemplate.query(
    "SELECT id, name, email, type FROM customer WHERE type = 'A'",
    (rs, rowNum) -> new Customer(
        rs.getLong("id"),
        rs.getString("name"),
        rs.getString("email"),
        rs.getString("type"))
);

assertFalse(dbCustomers.isEmpty());

dbCustomers.forEach(c -> {
    assertEquals(c.getName(), c.getName().toUpperCase());
    assertTrue(c.getEmail().startsWith("A_"));
});

Path outputFile = Paths.get("output/processed_customers.txt");
assertTrue(Files.exists(outputFile));

List<String> lines = Files.readAllLines(outputFile);

boolean hasTypeB = lines.stream().anyMatch(line -> line.endsWith(",B"));
assertTrue(hasTypeB);

lines.forEach(line -> {
    String[] parts = line.split(",");
    if ("B".equals(parts[3])) {
        assertEquals(parts[1], parts[1].toLowerCase());
        assertTrue(parts[2].startsWith("B_"));
    }
});
```

From the test case, we first query the database to check that customers of type *A* have been saved with their names converted to uppercase and their email prefixed with "A_". **Next, we also read the output file to confirm that customers of type *B* have their names converted to lowercase and their emails prefixed with "B_".**

9. Conclusion

In this article we learned how to configure a Spring Batch job using a single reader but multiple processors and writers. We read data from a CSV file, routed each record to a specific processor based on its content, and finally delegated the writing to multiple writers.

As always, the source code is available over on GitHub (<https://github.com/eugenp/tutorials/tree/master/spring-batch-2>).



Since its introduction in Java 8, the Stream API has become a staple of Java development. The basic operations like iterating, filtering, mapping sequences of elements are deceptively simple to use.

But these can also be overused and fall into some common pitfalls.

To **get a better understanding on how Streams work** and how to combine them with other language features, check out our guide to Java Streams:

>> Join Pro and download the eBook (/eBook-Java-Streams-NPI-EA-3-fcdg2)

Do JSON right with Jackson

Download the E-book (/ebook-jackson-NPI-EA-3pldr)

☒ Subscribe ▼



Be the First to Comment!

B *I* U



0 COMMENTS

COURSES

[ALL COURSES \(/COURSES/ALL-COURSES\)](/COURSES/ALL-COURSES)

[BAELDUNG ALL ACCESS \(/COURSES/ALL-ACCESS\)](/COURSES/ALL-ACCESS)

[BAELDUNG ALL TEAM ACCESS \(/COURSES/ALL-ACCESS-TEAM\)](/COURSES/ALL-ACCESS-TEAM)

[THE COURSES PLATFORM \(HTTPS://COURSES.BAELDUNG.COM\)](https://courses.baeldung.com)

SERIES

[JAVA "BACK TO BASICS" TUTORIAL \(/JAVA-TUTORIAL\)](/JAVA-TUTORIAL)

[LEARN SPRING BOOT SERIES \(/SPRING-BOOT\)](/SPRING-BOOT)

[SPRING TUTORIAL \(/SPRING-TUTORIAL\)](/SPRING-TUTORIAL)

[GET STARTED WITH JAVA \(/GET-STARTED-WITH-JAVA-SERIES\)](/GET-STARTED-WITH-JAVA-SERIES)

[SECURITY WITH SPRING \(/SECURITY-SPRING\)](/SECURITY-SPRING)

[REST WITH SPRING SERIES \(/REST-WITH-SPRING-SERIES\)](/REST-WITH-SPRING-SERIES)

[ALL ABOUT STRING IN JAVA \(/JAVA-STRING\)](/JAVA-STRING)

ABOUT

[ABOUT BAELDUNG \(/ABOUT\)](/ABOUT)

[THE FULL ARCHIVE \(/FULL_ARCHIVE\)](/FULL_ARCHIVE)

[EDITORS \(/EDITORS\)](#)

[OUR PARTNERS \(/PARTNERS/\)](#)



[PARTNER WITH BAELDUNG \(/PARTNERS/WORK-WITH-US\)](#)

[EBOOKS \(/LIBRARY/\)](#)

[FAQ \(/LIBRARY/FAQ\)](#)



[BAELDUNG PRO \(/MEMBERS/\)](#)

[TERMS OF SERVICE \(/TERMS-OF-SERVICE\)](#)

[PRIVACY POLICY \(/PRIVACY-POLICY\)](#)

[COMPANY INFO \(/BAELDUNG-COMPANY-INFO\)](#)

[CONTACT \(/CONTACT\)](#)

[PRIVACY MANAGER](#)